

```
context.ui = mock(BreakfastUI)
context.machine = BreakfastMachine(context.ui)
...
@then(u'there should be error message "{message}"')
def impl(context, message):
    verify(context.machine.ui).error(message)
```

We also need to modify our *BreakfastMachine* to connect to the user interface when it starts up. We do this by modifying the `__init__` method in *breakfast.py* as per the following:

```
def __init__(self, ui):
    super(BreakfastMachine, self).__init__()
    self.ui = ui
    self.eggs = 0
    self.egg_hardness = 'soft'
    self.eggs_to_boil = 0

    self.boiled_eggs = 0
    self.boiled_egg_hardness = None
```

There are two interesting bits in *eggs.py*. The first is in the method where we set the machine to stand by. Instead of using a real *BreakfastUI* object (which wouldn't have any implementation anyway), we create a test double that looks like *BreakfastUI*, but does not do anything when called. However, it can record all the calls, their parameters and order.

The second interesting part is the function where we verify that an error message has been delivered to the UI. We call *verify*, pass the UI object as a parameter to it, and then specify which method and parameters should be checked. Both *verify* and *mock* are part of Mockito, and offer us tools to check the interactions or the behaviour of objects.

If we run Behave after these modifications, we are going to get a new error message, as shown below:

```
Then there should be error message "not enough eggs" #
features\steps\eggs.py:35
Assertion Failed:
Wanted but not invoked: error(u'not enough eggs')
```

This tells us that the specification expected a call to the method *error*, with the parameter *'not enough eggs'*. However, our code does not currently do that, so the specification fails. Let's fix that and modify how the machine boils eggs (*breakfast.py*):

```
def boil_eggs(self):
    if self.eggs_to_boil <= self.eggs:
        self.boiled_eggs = self.eggs_to_boil
        self.boiled_egg_hardness = self.egg_hardness
```

```
self.eggs = self.eggs - self.eggs_to_boil
else:
    self.boiled_eggs = 0
    self.boiled_egg_hardness = None
    self.ui.error('not enough eggs')
```

Let's add a call to the UI object's *error* method in order to let the user interface know that there was an error, and that the user should be notified about it. After this modification, Behave should run again without errors and give us a summary:

```
1 feature passed, 0 failed, 0 skipped
2 scenarios passed, 0 failed, 0 skipped
12 steps passed, 0 failed, 0 skipped, 0 undefined
Took 0m0.0s
```

There is a distinction between these two approaches. In the first one, we wanted to verify the state of the system after it boiled the eggs. We checked that the amount of eggs boiled matched our specification, and that they were of the correct hardness.

In the second case, we did not even have a real user interface to work with. Instead of writing one from scratch in order to be able to run tests, we created a test double: an object that looks like a real user interface, but isn't one. With this double, we could verify that the breakfast machine calls the correct methods with the correct parameters.

Now we have the basics of boiling eggs. If you want, you can continue from here and add more features to the breakfast machine, like frying eggs and bacon, and squeezing juice out of oranges. Try and add a call to the user interface after the eggs have been boiled, and verify that you are sending the correct number of eggs and as per the specified hardness.

After getting comfortable with the approaches and techniques shown here, you can start learning more about Behave and Mockito-Python. Good places to start are their download pages, as both offer short tutorials.

In this brief example, we had a glimpse of several different ways of writing executable specifications. We started our project from a natural-language file that specified how we wanted our machine to boil eggs. After this specification, we wrote an implementation for each step, and used that to help us to write our actual breakfast machine code. After learning how to verify the state of a system, we switched our focus to verifying how objects behave and communicate with each other. We finished with a breakfast machine that can boil a given number of soft or hard-boiled eggs, and that will issue a notification to the user interface in case there are not enough eggs in the machine. 

By: Tuukka Turto

The author is a software aficionado.